



Project Development Guide

Smart Budget & Recommendation Assistant - End-to-End Implementation Blueprint

This document provides the complete technical development breakdown for **PocketSmart AI**. It details the full-stack architecture, including Frontend UI components, Backend REST API endpoints, Database schemas, and AI Integration logic required to build a seamless financial management assistant.

1. Full-Stack Technology Architecture

Layer	Technology Stack	Core Responsibilities
Frontend	React.js / Next.js, Tailwind CSS	Interactive user interface, real-time dashboard, budget visualization, responsive layout.
Backend API	Node.js / Express.js (or Python FastAPI)	Business logic, authentication (JWT), transaction processing, API routing.
Database	MongoDB (NoSQL) / PostgreSQL	User profiles, income/expense logs, budget targets, category classifications.
AI Engine	Gemini API / OpenAI API	Financial pattern analysis, personalized savings tips, budget anomaly detection.

2. Database Schema Design (MongoDB Model)

Structured schema for managing user accounts, income, and transaction records:

User & Transaction Schema Definition

DATABASE MODELS

```
// Users Schema
{
  _id: ObjectId,
  name: String,
  email: String (Unique),
  passwordHash: String,
  monthlyIncome: Number,
  createdAt: Date
}

// Transactions Schema
{
  _id: ObjectId,
  userId: ObjectId (Ref: User),
  title: String,
  amount: Number,
  category: String, // 'Food', 'Rent', 'Entertainment', 'Bills'
  type: String, // 'Income' or 'Expense'
  date: Date
}
```

3. Backend REST API Implementation

Express.js API Endpoint Example

BACKEND LOGIC (POST /API/TRANSACTIONS)

```
const express = require('express');
const router = express.Router();
const Transaction = require('../models/Transaction');

// Add New Transaction
router.post('/add', async (req, res) => {
  try {
    const { userId, title, amount, category, type } = req.body;
    const newTransaction = new Transaction({ userId, title, amount, category, type });
    await newTransaction.save();
    res.status(201).json({ success: true, data: newTransaction });
  } catch (error) {
    res.status(500).json({ success: false, message: error.message });
  }
});

module.exports = router;
```

4. AI Recommendation Integration

Gemini AI Assistant Integration

AI RECOMMENDATION ENDPOINT

```
const { GoogleGenerativeAI } = require('@google/generative-ai');
const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);

async function getBudgetRecommendation(income, expenses) {
  const model = genAI.getGenerativeModel({ model: "gemini-1.5-flash" });
  const prompt = `User Income: ${income}. Expenses: ${JSON.stringify(expenses)}.
  Provide 3 actionable tips to optimize budget in 2 short bullet points.`;

  const result = await model.generateContent(prompt);
  return result.response.text();
}
```

5. Frontend Component Implementation

React Dashboard Component

FRONTEND UI INTEGRATION

```
import React, { useState, useEffect } from 'react';

export default function BudgetSummary({ income, totalExpenses }) {
  const remainingBalance = income - totalExpenses;

  return (
    <div className="p-4 bg-white rounded-xl shadow-md border border-slate-200">
      <h3 className="text-lg font-bold text-slate-800">Monthly Overview</h3>
      <div className="mt-2 grid grid-cols-2 gap-4">
        <div className="p-2 bg-green-50 text-green-700 rounded">Income: ${income}</div>
        <div className="p-2 bg-red-50 text-red-700 rounded">Expenses: ${totalExpenses}</div>
      </div>
      <div className="mt-3 text-md font-semibold text-blue-800">
        Balance: ${remainingBalance}
      </div>
    </div>
  );
}
```